RUNI – Algorithms – Homework 7

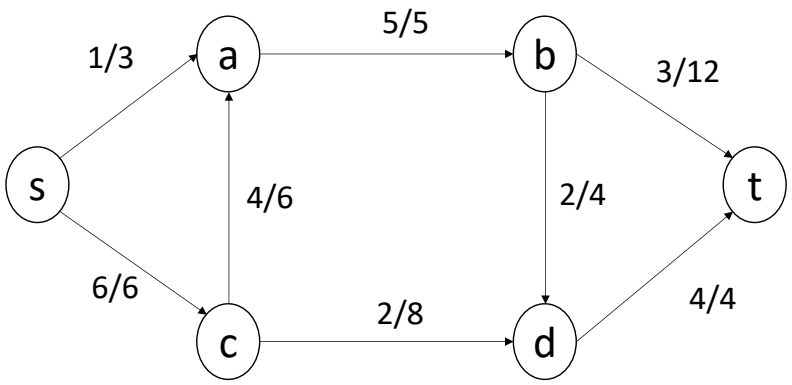
Due date: Sunday 18/01/2026

NOTE: the general homework guidelines published in homework 1 are valid for all homework assignments in this course.

Note that the due-date is more than two weeks away. It is recommended to start early, however, be aware that we will cover the time complexity analysis of max-flow algorithms as well as the material required to solve Problems 4 and 5 only next week. These advanced tasks are marked with ♠.

Problem 1 (20 pts):

The figure presents a flow network with a legal flow function. The given flow is a result of **three iterations** of Ford & Fulkerson algorithm.



Answer the following, no need to justify your answers.

a. The augmenting path the was implemented in the first iteration is

b. Suggest an augmenting path for the next (4-th) iteration, to improve the flow.

c. What is the value of the maximum flow in the network?

d. Write all sets of nodes inducing a min-cut in the network.

Problem 2 (20 pts):

Complete the following algorithm that decides if a given network $G = (V, E, c)$ has only a single minimum $s - t$ cut. Analyze its time complexity (♠) and justify its correctness.

Algorithm:

1. Run the Edmunds-Karp algorithm to find the maximum flow f. Let N_f be the residual network.

Problem 3 (21 pts)

We are given a network in which all capacities are integers and the max-flow in the network is known to be $F = X > 0$. The capacity of some given edge e is increased. The max-flow in the resulting network is now Y. Prove or give a counter example (for each statement separately):

a. If, before the change, e belong to some min-cut then $X < Y$.

b. If e is upward critical (as defined in the recitation) then $c(e)$ was raised by $Y - X$.

c. If $Y = 2X$ then in the original max-flow e was saturated.

♠Problem 4 (15 pts)

Consider the maximum matching problem in a bipartite graph. The input is given as an undirected bipartite G=(V,E), where $V=V_1 \cup V_2$, $V_1=\{b_1,\ldots,b_{n1}\}$, $V_2=\{g_1,\ldots,g_{n2}\}$ and $E \subseteq V_1 \times V_2$.

Definition: An edge $(b_i,g_j) \in E$ is **redundant** if the two graphs G=(V,E) and G'=(V,E-{(b_i,g_j)}) have maximum matchings of the same size.

Suggest an efficient algorithm that returns all the redundant edges in G. Justify the correctness of your algorithm and analyze its time complexity.

♠Problem 5 (24 pts)

After a historic year of growth, Nvidia has announced the construction of a new flagship R&D campus in Kiryat Tivon, inspired by the company's California headquarters and expected to become a centerpiece of the northern Israeli tech ecosystem.

To staff the campus, Nvidia decided to launch an accelerated hiring program focused on candidates who have demonstrated strong analytical thinking reflected across multiple evaluation criteria, including outstanding performance in their Algorithms course.

Rather than relying solely on interviews, Nvidia's hiring committee aims to place suitable candidates directly into a precise assignment of roles and teams, while respecting strict internal capacity and qualification constraints.

There are four disjoint sets:

- C – Candidates waiting for an assignment.
- T – development teams at the Tivon campus;
- R – senior responsibility roles (e.g. architecture lead, principal performance engineer).

Each candidate from C may be assigned to at most one team and at most one role.

Each team, and each role, may be assigned at most one new candidate.

For every candidate$c \in C$, Nvidia provides:

- A set $T(c) \subseteq T$ of teams which suit the candidate's overall set of skills;
- A set $R(c) \subseteq R$ of roles that the candidate qualifies for.

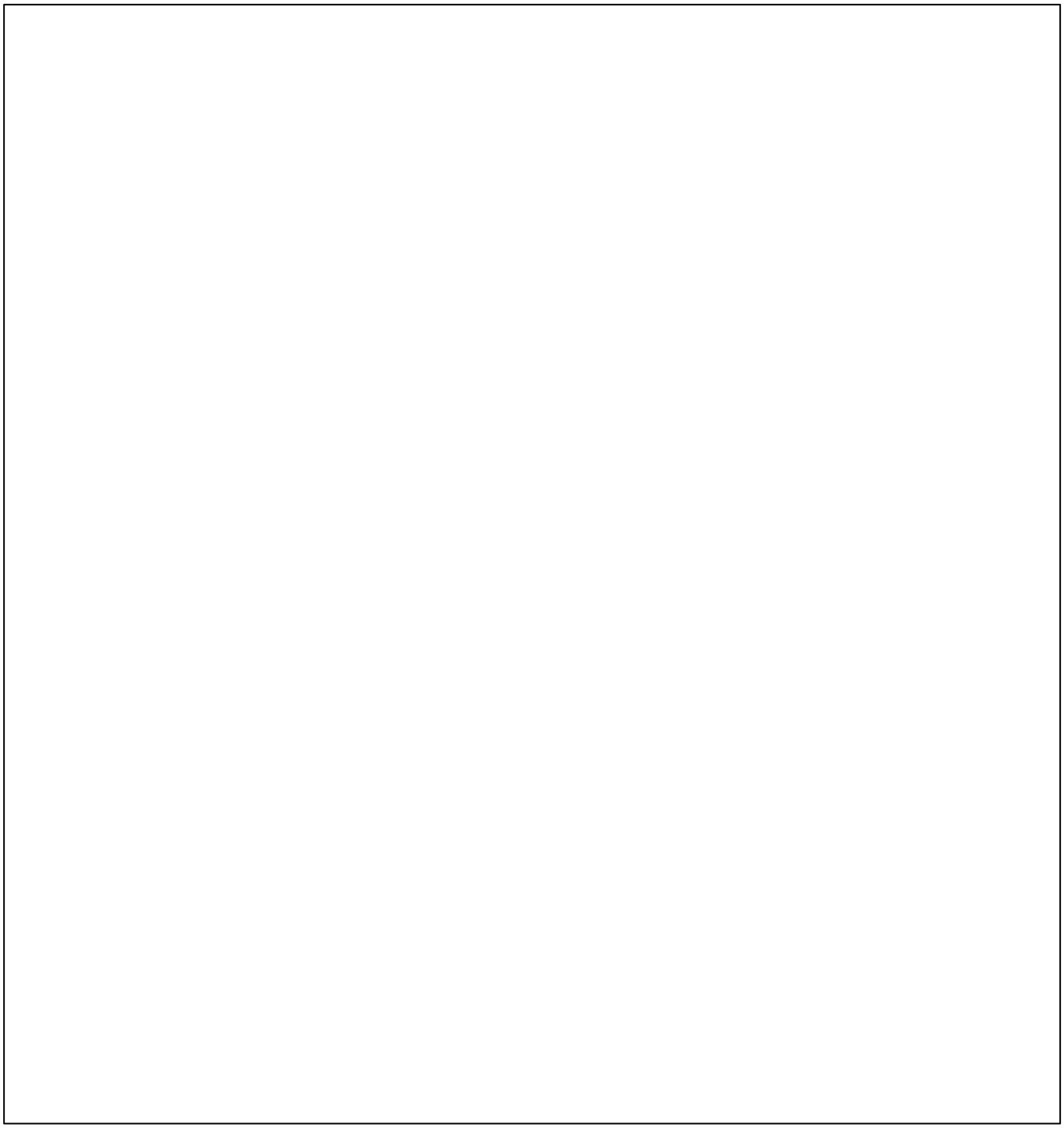
a. Performance-Based Flexibility Hiring. In its first hiring attempt, Nvidia allowed flexibility for the set C of candidates. A candidate may fall into one of the following categories:

1. Not Hired.
2. Assigned to exactly one team.
3. Assigned to exactly one role.
4. Assigned to both one team and one role.

The objective of this hiring stage is to **maximize the number of filled positions**.

Design an efficient algorithm that finds an optimal assignment under this hiring policy.

Analyze the time complexity of the algorithm, and justify shortly. No need to prove your algorithm.



b. Strict Position and Team Hiring. Following an internal audit, Nvidia concluded that the positions must be filled only by those candidates that are strong enough to justify both their role's technical level and the team's specialty. Thus, under this hiring policy a candidate may only fall into either:

1. Not hired.
2. Assigned simultaneously to one team and one role.

Assignments to only a position or only a role are not permitted.

The objective of this hiring stage again is to **maximize the number of filled positions**.

Design an efficient algorithm that finds an optimal assignment under this hiring policy. Prove the algorithm's correctness (provide formal main claim and prove it). No need to analyze complexity.